

Chapter2 RTL Design and Verification

2.1 CPU Design

Table 8 difference between ISA and Microarchitecture

ISA (architecture)	Microarchitecture
<i>What the CPU does (programmer's view)</i>	<i>How the CPU actually implements it (hardware designer's view)</i>
Defines: Instructions Registers Data types Instruction formats Memory model Addressing modes	Defines: Pipeline design (5 stage, 7 stage) Hazard handling Cache design Branch prediction logic Execution unit (ALU , MUX)
Visible to programmer	Not visible to user
Focuses on functionality	Focuses on implementation
Eg: RISC , CISC	Eg RISC V pipeline CPU , single cycle

RISC-V is an **open-standard Instruction Set Architecture (ISA)** based on the **Reduced Instruction Set Computer (RISC)** philosophy.

Key Characteristics

- Open and royalty-free
- Simple and modular design
- Designed for performance, scalability, and teaching

Benefits:

- No licensing fees
- No legal restrictions
- Complete visibility of specifications
- Enables custom extensions

RISC V Extensions include:

<i>Extension</i>	Function
<i>M</i>	Multiply/Divide
<i>F</i>	Floating point
<i>C</i>	Compressed instructions
<i>A</i>	Atomic operations
<i>I</i>	Integers

RV32I Specifications:

RV32I is **the** base integer instruction set **of RISC-V**. Meaning of RV32I:

- **RV** → RISC-V
- **32** → 32-bit architecture (registers are 32 bits wide)
- **I** → Base **I**nteger instruction set

RV32I follows core RISC principles:

1. Simplicity

- Few instruction formats
- Fixed instruction size (32 bits)

2. Load/Store Architecture

- Memory accessed **only by load/store instructions**
- ALU operations use **registers only**

3. Regularity

- Consistent instruction encoding
- Easier hardware implementation

a) Register file –RV32I has 32 General purpose Registers

Register	Name	Purpose
x0	zero	Always 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5–x7	t0–t2	Temporaries
x8–x9	s0–s1	Saved registers
x10–x17	a0–a7	Arguments/returns

Each register is 32 bit wide.

Memory size – $32 \times 32 = 1024 \text{ bits} = 1 \text{ KB}$

To address these 32 registers ,5 bit address is required eg.rs1 rs2,rd.

b) Instruction Formats – fixed 32 bit instructions with formatting

Table 9 Instruction types

R type (register)	Add, sub, and , or, xor, sll, srl,sra
I type (immediate)	addi , lw, ori, andi
S type (store)	sw
B type (Branch)	beq, bne, blt,bge
U type (upper immediate)	lui
J type (jump)	Jal , jalr

RISC-V encoding is designed so hardware can quickly identify instruction type and operands using fixed bit positions.

31	[30 — 25]	[24 - 21]	20	[19 - 15]	[14 — 12]	[11 — 8]	7	[6 — 0]	
funct7		rs2		rs1	funct3		rd	opcode	R - type
imm [11:0]				rs1	funct3		rd	opcode	I - type
imm [11:5]		rs2		rs1	funct3	imm [4:0]		opcode	S - type
imm [12]	imm [10:5]	rs2		rs1	funct3	imm [4:1]	imm [1]	opcode	B - type
imm [31:12]						rd		opcode	U - type
imm [20]	imm [10:1]		imm [1]	imm [19:12]		rd		opcode	J - type

Figure 2 RISC V instruction Encoding

Table 10 RV32I instructions with Formats

R type	Operation	Opcode	funct3	funct7
add	$rd = rs1 + rs2$	0110011	000	0000000
sub	$rd = rs1 - rs2$	0110011	000	0100000
xor	$rd = rs1 \wedge rs2$	0110011	100	0000000
or	$rd = rs1 \mid rs2$	0110011	110	0000000
and	$rd = rs1 \& rs2$	0110011	111	0000000
sll	$rd = rs1 \ll rs2$	0110011	001	0000000
srl	$rd = rs1 \gg rs2$ (logical)	0110011	101	0000000
sra	$rd = rs1 \gg rs2$ (arith)	0110011	101	0100000
slt	$rd = (rs1 < rs2)$	0110011	010	0000000
sltu	$rd = (rs1 < rs2 \text{ unsigned})$	0110011	011	0000000
I type	Operation	Opcode	Funct3	immediate
Lb	$rd = M[rs1 + imm]$	0000011	000	Imm[11:0]
Lh	$rd = M[rs1 + imm]$	0000011	001	Imm[11:0]
Lw	$rd = M[rs1 + imm]$	0000011	010	Imm[11:0]
Ibu	$rd = M[rs1 + imm]$ (unsigned)	0000011	100	Imm[11:0]
lhu	$rd = M[rs1 + imm]$ (unsigned)	0000011	101	Imm[11:0]
S type	Operation	Opcode	Funct3	immediate
Sb	$M[rs1 + imm] = rs2$	0100011	000	Imm[11:5]
Sh	$M[rs1 + imm] = rs2$	0100011	001	Imm[11:5]
sw	$M[rs1 + imm] = rs2$	0100011	010	Imm[11:5]
B type	Operation	Opcode	Funct3	immediate
Beq	$rs1 == rs2$	1100011	000	SXT{Imm[12:1],1'b0}
Bne	$rs1 \neq rs2$	1100011	001	SXT{Imm[12:1],1'b0}
Bltn	$rs1 < rs2$	1100011	100	SXT{Imm[12:1],1'b0}
Bge	$rs1 \geq rs2$	1100011	101	SXT{Imm[12:1],1'b0}
Bltn	$rs1 < rs2 \text{ (unsigned)}$	1100011	110	SXT{Imm[12:1],1'b0}
bgeu	$rs1 \geq rs2 \text{ (unsigned)}$	1100011	111	SXT{Imm[12:1],1'b0}
J type	Operation	Opcode	Funct3	immediate
Jal	$rd = PC + 4; PC += imm$	1101111	NA	{Imm[20:1],1'b0}
Jalr	$rd = PC + 4; PC += imm$	1101111	000	Imm[11:0]
U type	Operation	Opcode	Funct3	immediate
Lui	$rd = imm \ll 12$	0110111	NA	{Imm[31:12],12'b0}
auipc	$rd = PC + (imm \ll 12)$	0010111	NA	{Imm[31:12],12'b0}

c) Data memory in RV32I:

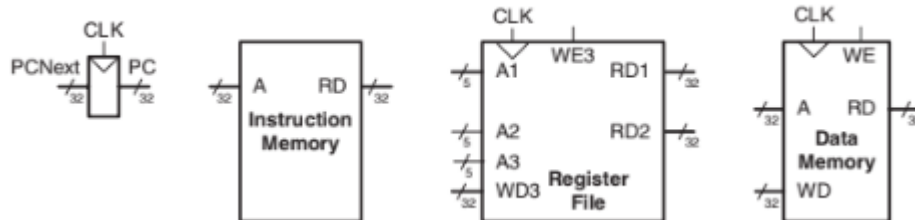
A byte addressable memory which Store and retrieve program data (variables, arrays, stack, heap). It is separate in function from instruction memory (which stores code), even if physically unified.

Each address refers to 1 byte. A word size of 32 bit is supported. (1 word = 4Bytes = 32bit).

Only lw/sw instructions access this memory.

2.3.9 Single Cycle Microarchitecture of 32 bit RISC V CPU

We first design a microarchitecture that executes instructions in a single cycle. We begin constructing the datapath by connecting the state elements from Figure 7.1 with combinational logic that can execute the various instructions. Control signals determine which specific instruction is performed by the datapath at any given time. The control unit contains combinational logic that generates the appropriate control signals based on the current instruction. Finally, we analyze the performance of the single-cycle processor.



This section gradually develops the single-cycle datapath, adding one piece at a time to the state elements from Figure 7.1. The new connections are emphasized in black (or blue, for new control signals), whereas the hardware that has already been studied is shown in gray. The example instruction being executed is shown at the bottom of each figure. The program counter contains the address of the instruction to execute. The first step is to read this instruction from instruction memory. Figure 7.3 shows that the PC is simply connected to the address input of the instruction memory. The instruction memory reads out, or fetches, the 32-bit instruction, labeled Instr. In our sample program from Figure 7.2, PC is 0x1000. (Note that this is a 32-bit processor, so PC is really 0x00001000, but we omit leading zeros to avoid

cluttering the figure)

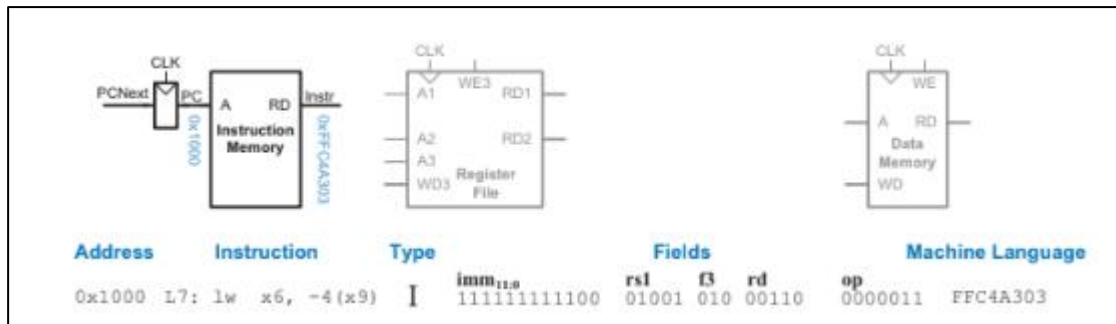


Figure 3 Fetch instruction from memory

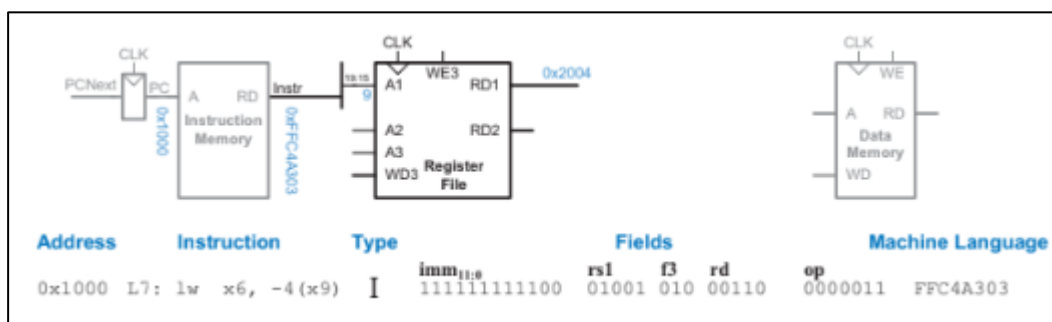


Figure 4 Read source operand from register file

Instr is lw, 0xFFC4A303, as also shown at the bottom of Figure 7.3. These sample values are annotated in light blue on the diagram. The processor's actions depend on the specific instruction that was fetched. First, we will work out the datapath connections for the lw instruction. Then, we will consider how to generalize the datapath to handle other instructions.

Lw:

For the lw instruction, the next step is to read the source register containing the base address. Recall that lw is an I-type instruction, and the base register is specified in the rs1 field of the instruction, Instr_{19:15}. These bits of the instruction connect to the A1 address input of the register file, as shown in Figure 7.4. The register file reads the register value onto RD1. In our example, the register file reads 0x2004 from x9.

The lw instruction also requires an offset. The offset is stored in the 12-bit immediate field of the instruction, Instr_{31:20}. It is a signed value, so it must be sign-extended to 32 bits. Sign extension simply means copying the sign bit into the most significant bits: ImmExt_{31:12} = Instr₃₁, and ImmExt_{11:0} = Instr_{31:20}. Sign-extension is performed by an Extend unit, as shown in Figure 7.5, which receives the 12-bit signed immediate in Instr_{31:20} and produces the 32-bit sign-extended immediate, ImmExt. In our example, the two's complement immediate -4 is extended from its 12-bit representation 0xFFC to a 32-bit representation 0xFFFFFFFCC.

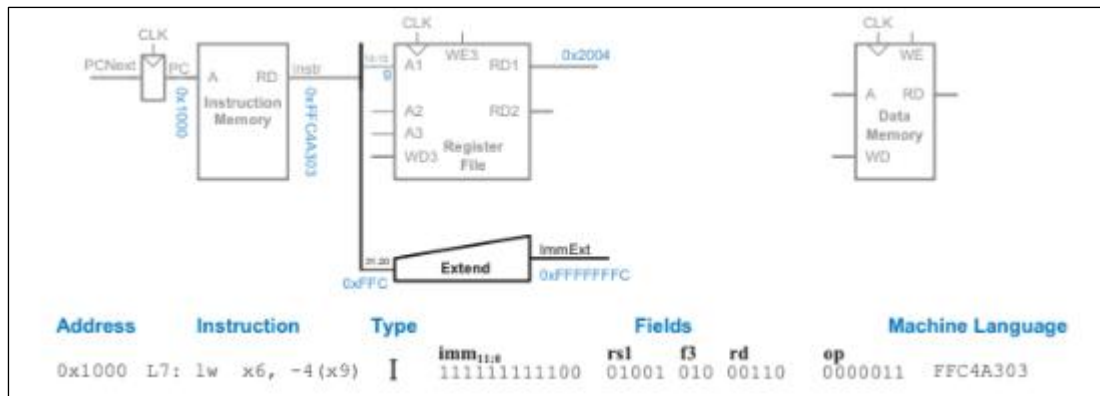


Figure 5 Sign Extend the Immediate

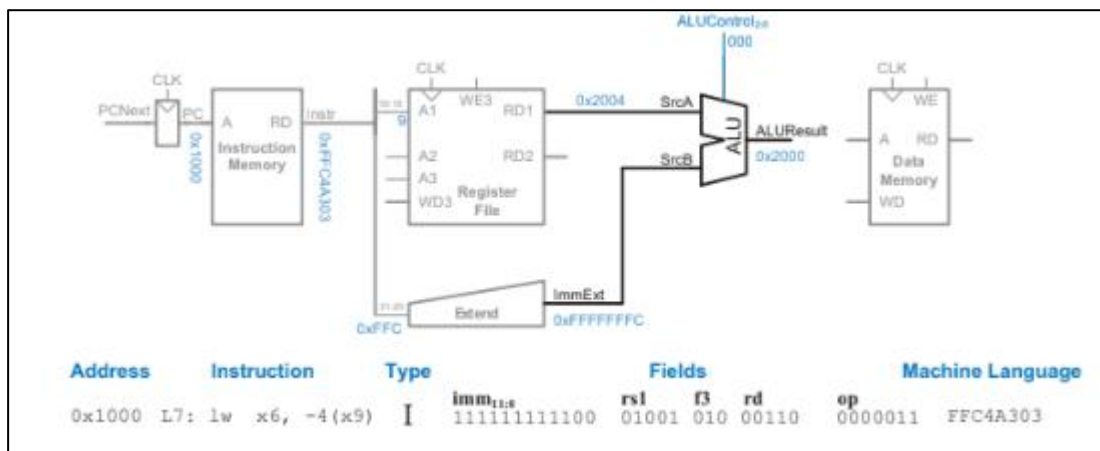


Figure 6 Compute memory address

The processor adds the base address to the offset to find the address to read from memory. Figure 7.6 introduces an ALU to perform this addition. The ALU receives two operands, SrcA and SrcB. SrcA is the base address from the register file, and SrcB is the offset from the sign-extended immediate, ImmExt. The ALU can perform many operations, as was described in Section 5.2.4. The 3-bit ALUControl signal specifies the operation (see Table 5.3 on page 250). The ALU receives 32-bit operands and generates a 32-bit ALUResult. For the lw instruction, ALUControl should be set to 000 to perform addition. ALUResult is sent to the data memory as the address to read, as shown in Figure 7.6.

In our example, the ALU computes $0x2004 + 0xFFFFF0FC = 0x2000$. Again, this is a 32-bit value, but we omit the leading zeros to avoid cluttering the figure.

This memory address from the ALU is provided to the address (A) port of the data memory. The data is read from the data memory onto the ReadData bus and then written back to the destination register at the end of the cycle, as shown in Figure 7.7. Port 3 of the register file is the write port. lw's destination register, indicated by the rd field (Instr11:7), is connected to A3, port 3's address input. The ReadData bus is connected to WD3, port 3's write data input. A control signal called RegWrite (register write) is connected to WE3, port 3's write enable input,

and is asserted during the lw instruction so that the data value is written into the register file. The write takes place on the rising edge of the clock at the end of the cycle. In our example, the processor reads 10 from address 0x2000 in the data memory and puts that value (10) into x6 in the register file.

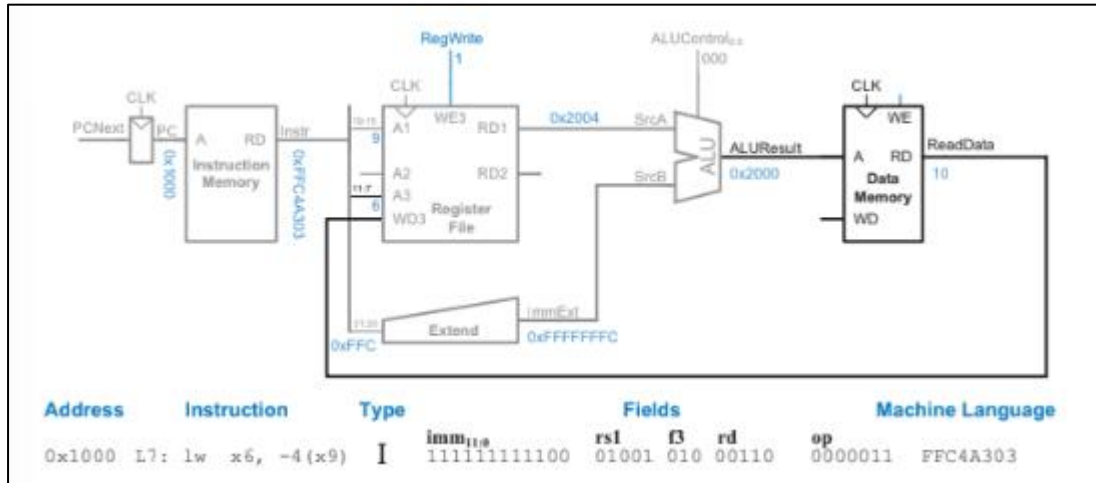


Figure 7 Read memory and write result back to Register file

While the instruction is being executed, the processor must also compute the address of the next instruction, PCNext. Because instructions are 32 bits (4 bytes), the next instruction is at PC+4. Figure 7.8 uses an adder to increment the PC by 4. In our example, PCNext = 0x1000 + 4 = 0x1004. The new address is written into the program counter on the next rising edge of the clock. This completes the datapath for the lw instruction.

Sw:

Next, let us extend the datapath to handle sw, which is an S-type instruction. Like lw, sw reads a base address from port 1 of the register file and sign-extends the immediate. The ALU adds the base address to the immediate to find the memory address. All of these functions are already supported in the datapath, but the 12-bit signed immediate is stored in Instr31:25,11:7 (instead of Instr31:20, as it was for lw). Thus, the Extend unit must be modified to also receive these additional bits, Instr11:7. For simplicity (and for future instructions such as jal), the Extend unit receives all the bits of Instr31:7. A control signal, ImmSrc, decides which instruction bits to use as the immediate. When ImmSrc = 0 (for lw), the Extend unit chooses Instr31:20 as the 12-bit signed immediate; when ImmSrc = 1 (for sw), it chooses Instr31:25,11:7.

The sw instruction also reads a second register from the register file and writes its contents to the data memory. Figure 7.9 shows the new connections for this added functionality. The register is specified in the rs2 field, Instr24:20, which is connected to the address 2 (A2) input of the register file. The register's contents are read onto the read data 2 (RD2) output, which, in turn, is connected to the write data (WD) input of the data memory. The write enable port of the data memory, WE, is controlled by MemWrite. For an sw instruction: MemWrite = 1 to write the data to memory; ALUControl = 000 to add the base address and offset; and RegWrite

= 0, because nothing should be written to the register file. Note that data is still read from the address given to the data memory, but this ReadData is ignored because RegWrite = 0.

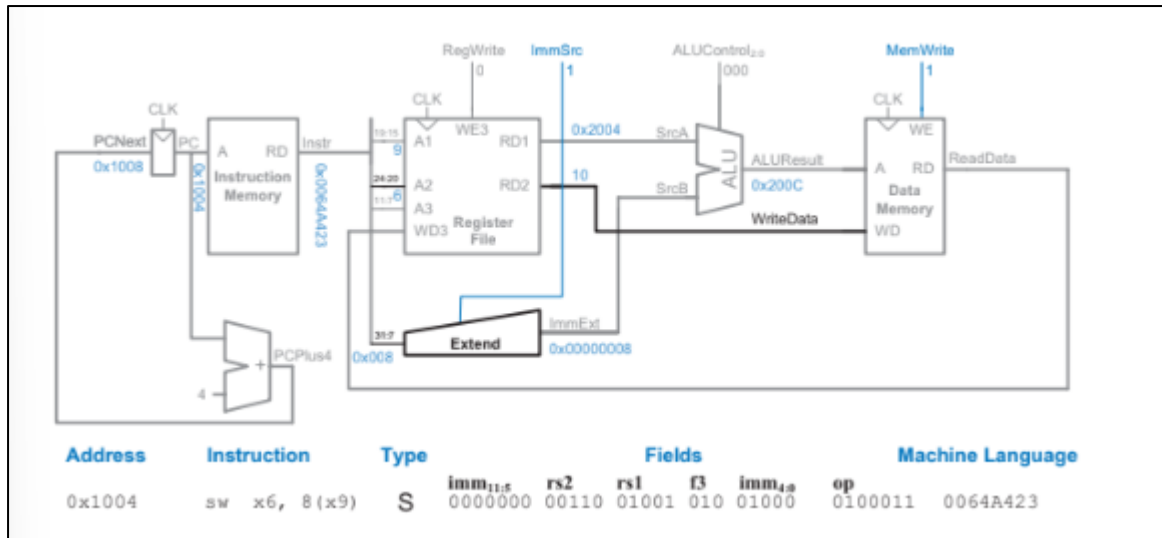


Figure 8 write data to memory for sw instruction

In our example, the PC is 0x1004. Thus, the instruction memory reads out the sw instruction, 0x0064A423. The register file reads 0x2004 (the base address) from x9 and 10 from x6 while the Extend unit extends the immediate offset 8 from 12 to 32 bits. The ALU computes $0x2004 + 8 = 0x200C$. The data memory writes 10 to address 0x200C. Meanwhile, the PC is incremented to 0x1008.

R-Type Instructions:

Next, consider extending the datapath to handle the R-type instructions, add, sub, and, or, and slt. All of these instructions read two source registers from the register file, perform some ALU operation on them, and write the result back to the destination register. They differ only in the specific ALU operation. Hence, they can all be handled with the same hardware but with different ALUControl signals. ALUControl is 000 for addition, 001 for subtraction, 010 for AND, 011 for OR, and 101 for set less than.

Figure 7.10 shows the enhanced datapath handling these R-type instructions. The datapath reads rs1 and rs2 from ports 1 and 2 of the register file and performs an ALU operation on them. We introduce a multiplexer and a new select signal, ALUSrc, to select between ImmExt and RD2 as the second ALU source, SrcB. For lw and sw, ALUSrc is 1 to select ImmExt; for R-type instructions, ALUSrc is 0 to select the register file output RD2 as SrcB. Let us name the value to be written back to the register file Result. For lw, Result comes from the ReadData output of the memory. However, for R-type instructions, Result comes from the ALUResult output of the ALU. We add the Result multiplexer to choose the proper Result based on the type of instruction.

The multiplexer select signal ResultSrc is 0 for R-type instructions to choose ALUResult as Result; ResultSrc is 1 for lw to choose ReadData. We do not care about the value of ResultSrc for sw because it does not write the register file.

In our example, the PC is 0x1008. Thus, the instruction memory reads out the or instruction 0x0062E233. The register file reads source operands 6 from x5 and 10 from x6. ALUControl is 011, so the ALU computes $6 \mid 10 = 01102 \mid 10102 = 11102 = 14$. The result is written back to x4. Meanwhile, the PC is incremented to 0x100C.

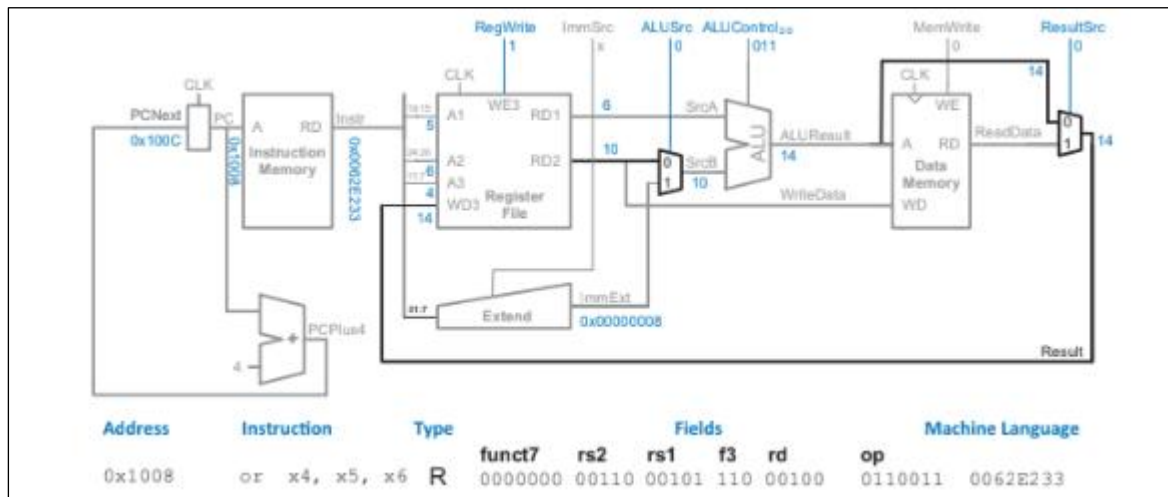


Figure 9 Datapath for R type

Beq:

Finally, we extend the datapath to handle the branch if equal (beq) instruction. beq compares two registers. If they are equal, it takes the branch by adding the branch offset to the program counter (PC). The branch offset is a 13-bit signed immediate stored in the 12-bit immediate field of the B-type instruction. Thus, the Extend logic needs yet another mode to choose the proper immediate. ImmSrc is increased to 2 bits, using the encoding Table. ImmExt is now either the sign-extended immediate (when ImmSrc = 00 or 01) or the branch offset (when ImmSrc = 10).

shows the modifications to the datapath. We need another adder to compute the branch target address, $PCTarget = PC + ImmExt$. The two source registers are compared by computing $(SrcA - SrcB)$ using the ALU. If ALUResult is 0, as indicated by the ALU's Zero flag, the registers are equal. We add a multiplexer to choose PCNext from either PCPlus4 or PCTarget. PCTarget is selected if the instruction is a branch and the Zero flag is asserted. For beq, $ALUControl = 001$, so that the ALU performs a subtraction. $ALUSrc = 0$ to choose SrcB from the register file. RegWrite and MemWrite are 0, because a branch does not write to the register file or memory. We don't care about the value of ResultSrc, because the register file is not written.

In our example, the PC is 0x100C, so the instruction memory reads out the beq instruction 0xFE420AE3. Both source registers are x4, so the register file reads 14 on both ports. The ALU computes $14 - 14 = 0$, and the Zero flag is asserted. Meanwhile, the Extend unit produces 0xFFFFFFF4 (i.e., -12), which is added to PC to obtain $PCTarget = 0x1000$. Note that we show the unswizzled upper 12 bits of the 13-bit immediate on the input of the Extend unit (0xFFA).

The PCNext mux chooses PCTarget as the next PC and branches back to the start of the code at the next clock edge.

This completes the design of the single-cycle processor datapath. We have illustrated not only the design itself but also the design process in which the state elements are identified and the combinational logic is systematically added to connect the state elements. In the next section, we consider how to compute the control signals that direct the operation of our datapath.

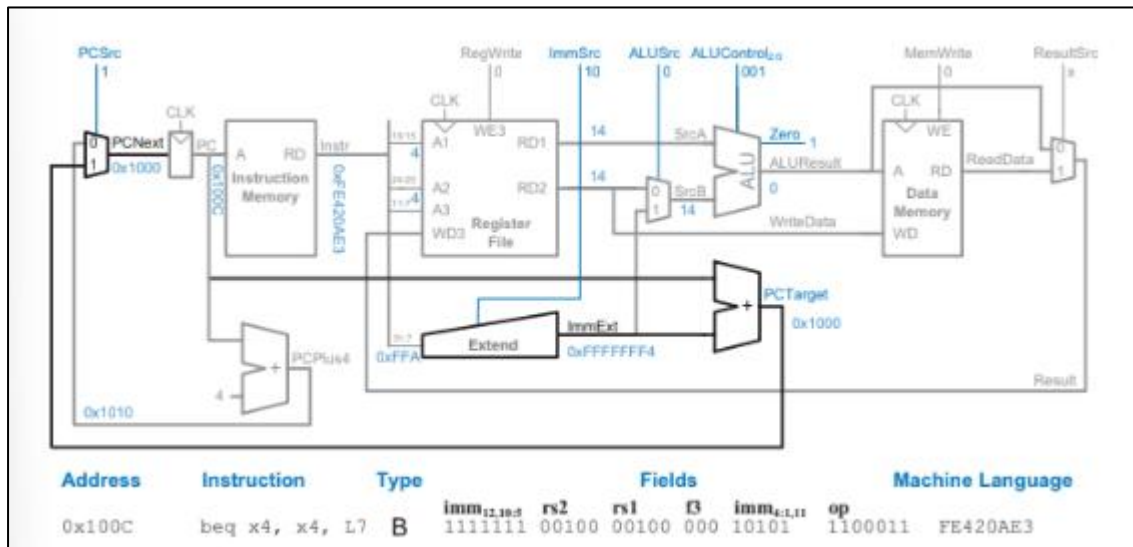


Figure 10 Datapath for branch type instructions

Jal:

The processor calculates the jump target address, the value of PCNext, by adding PC to the 21-bit signed immediate encoded in the instruction. The least significant bit of the immediate is always 0 and the next 20 most significant bits come from Instr_{31:12}. This 21-bit immediate is then sign-extended. The datapath already has hardware for adding PC to a sign-extended immediate, selecting this as the next PC, computing PC+4, and writing a value to the register file. Hence, in the datapath, we must only modify the Extend unit to sign-extend the 21-bit immediate and expand the Result multiplexer to choose PC+4.

The control unit needs to set PCSrc = 1 for the jump. To do this, we add an OR gate and another control signal, Jump. When Jump asserts, PCSrc = 1 and PCTarget (the jump target address) is selected as the next PC.

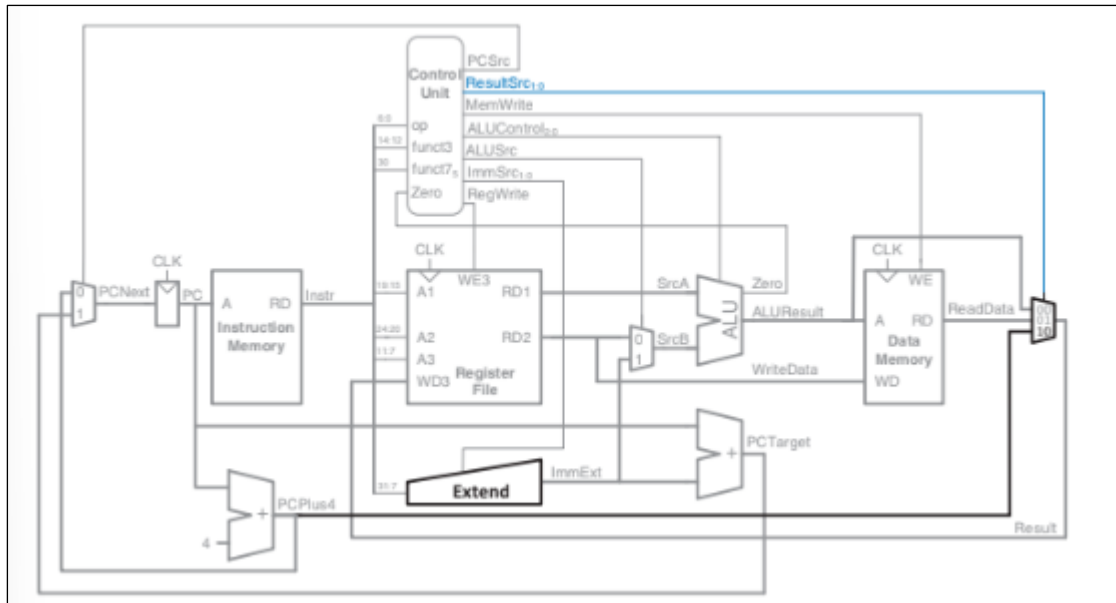


Figure 11 Complete Single Cycle Datapath

Single Cycle Control:

The single-cycle processor's control unit computes the control signal based on `op`, `funct3`, and `funct7`. For the RV32I instruction set, only bit 5 of `funct7` is used, so we just need to consider `op` (`Instr6:0`), `funct3` (`Instr14:12`), and `funct75` (`Instr30`).

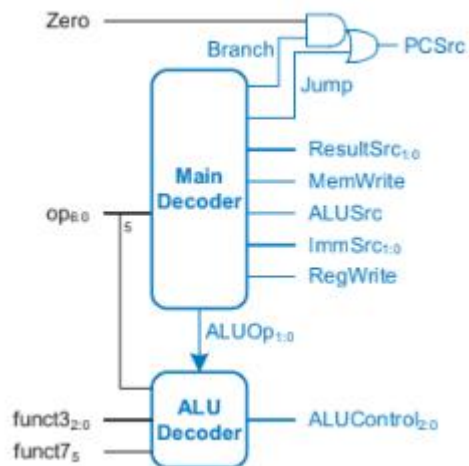


Figure 12 Completer Control Path for Singel Cycle

Figure 7.13 hierarchically decomposes the control unit, which is also referred to as the controller or the decoder, because it decodes what the instruction should do. We partition it into two major parts: the Main Decoder, which produces most of the control signals, and the ALU Decoder, which determines what operation the ALU performs.

Table 7.2 shows the control signals that the Main Decoder produces, as we determined while designing the datapath. The Main Decoder determines the instruction type from the opcode and then produces the appropriate control signals for the datapath. The Main Decoder generates most of the control signals for the datapath. It also produces internal signals Branch and ALUOp, signals used within the controller. The logic for the Main Decoder can be developed from the truth table using your favorite techniques for combinational logic design. The ALU Decoder produces ALUControl based on ALUOp and funct3.

Instruction	Opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
lw	0000011	1	00	1	0	01	0	00	0
sw	0100011	0	01	1	1	xx	0	00	0
R-type	0110011	1	xx	0	0	00	0	10	0
beq	1100011	0	10	0	0	xx	1	01	0
I-type ALU	0010011	1	00	1	0	00	0	10	0
jal	1101111	1	11	x	0	10	0	xx	1

Figure 13 Main decoder truth table

RTL Design way to Proceed

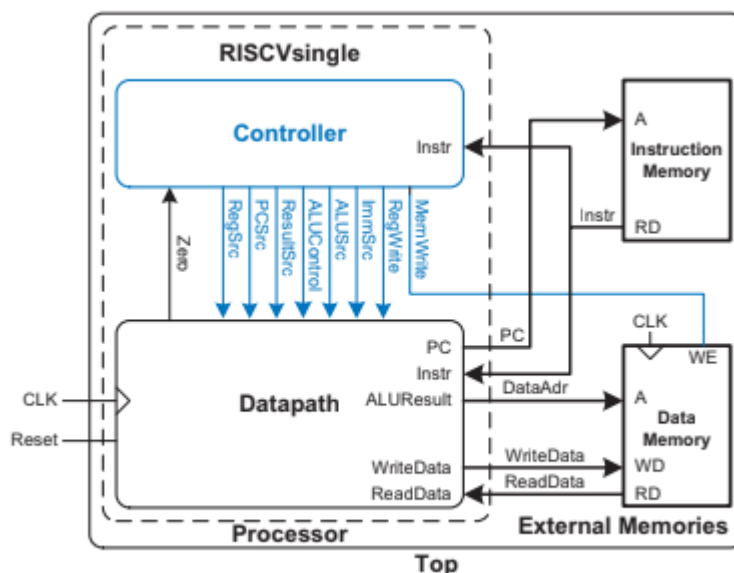


Figure 14 Single Cycle RISC V CPU interfaced to External Memories

- First start building Datapath and Controller Path module in Verilog/System Verilog
- Define Instruction memory and fill the machine codes in it.
- Then, define Data memory
- Start instantiating controller module and Datapath module in “ riscvingle” module .
- In the TOP level module, instantiate riscvsingle, Instruction memory and Data memory modules as shown in figure 14.
- Finally develop Testbench to test DUT(Top level module) and inspect the waveform and performance of CPU.

2.2.1 Pipelined Processor

We design a pipelined processor by subdividing the single-cycle processor into five pipeline stages. Thus, five instructions can execute simultaneously, one in each stage. Because each stage has only one-fifth of the entire logic, the clock frequency is approximately five times faster. So, ideally, the latency of each instruction is unchanged, but the throughput is five times better. Microprocessors execute millions or billions of instructions per second, so throughput is more important than latency. Pipelining introduces some overhead, so the throughput will not be as high as we might ideally desire, but pipelining nevertheless gives such great advantage for so little cost that all modern high-performance microprocessors are pipelined.

Reading and writing the memory and register file and using the ALU typically constitute the biggest delays in the processor. We choose five pipeline stages so that each stage involves exactly one of these slow steps. Specifically, we call the five stages Fetch, Decode, Execute, Memory, and Writeback. They are similar to the five steps that the multicycle processor used to perform lw. In the Fetch stage, the processor reads the instruction from instruction memory. In the Decode stage, the processor reads source operands from the register file and decodes the instruction to produce the control signals. In the Execute stage, the processor performs a computation with the ALU. In the Memory stage, the processor reads or writes data memory, if applicable. Finally, in the Writeback stage, the processor writes the result to the register file, if applicable.

Pipeline Datapath:

The pipelined datapath is formed by chopping the single-cycle datapath into five stages separated by pipeline registers. Figure 15 shows the pipelined datapath formed by inserting four pipeline registers to separate the datapath into five stages. The stages and their boundaries are indicated in blue. Signals are given a suffix (F, D, E, M, or W) to indicate the stage in which they reside. The register file is peculiar because it is read in the Decode stage and written in the Writeback stage. So, although the register file is drawn in the Decode stage, its write address and write data come from the Writeback stage. This feedback will lead to pipeline hazards. The register file in the pipelined processor writes on the falling edge of CLK so that it can write a result in the first half of a cycle and read that result in the second half of the cycle for use in a subsequent instruction. One of the subtle but critical issues in pipelining is that all signals associated with a particular instruction must advance through the pipeline in unison. Figure 15 has an error related to this issue. Can you find it? The error is in the register file write logic, which should operate in the Writeback stage. The data value comes from ResultW, a Writeback